

Université des Sciences et de la Technologie d'Oran – Mohamed Boudiaf

Faculté des Mathématiques et Informatique | Département
d'Informatique

MASTER 1 - Systèmes d'Information et Données (SID)

Tri distribué avec le simulateur GridSim

Rapport de Travaux Pratiques

Module : **Grilles informatiques**

Étudiants

Bekkouche Tayeb
Bakkar Ilyes Abdelillah
Benarbia Bouabdallah

Groupe : 2

Encadrement

Mme S. SENHADJI

Année : 2025-2026

Résumé

Ce rapport présente une implémentation d'un tri distribué dans un environnement de grille informatique simulé à l'aide du simulateur GridSim. Une architecture maître-travailleurs en topologie en étoile est utilisée pour répartir un ensemble de N valeurs numériques non triées entre plusieurs nœuds de calcul. Chaque travailleur trie localement sa sous-liste, puis le maître fusionne les résultats partiels à l'aide d'un algorithme de fusion k -chemins efficace basé sur une file de priorité.

L'objectif de ce TP est de démontrer les avantages du traitement parallèle et de la distribution des tâches dans les grilles informatiques. Les résultats montrent une bonne répartition de la charge et une fusion correcte produisant une liste finale triée.

Mots-clés : Tri distribué, GridSim, Architecture Maître-Travailleurs, Grilles informatiques, Fusion k -chemins.

Table des Matières

1	Introduction	3
2	Méthode de tri distribué	3
3	Topologie de la grille	3
4	Rôle des entités	4
5	Stratégie de distribution et fusion	4
6	Résultats de simulation	5
7	Discussion	5
8	Conclusion	6

1 Introduction

L'informatique en grille (*Grid Computing*) désigne une infrastructure distribuée permettant de mutualiser des ressources de calcul, de stockage et de communication hétérogènes à grande échelle. Contrairement aux systèmes centralisés, les grilles permettent d'exploiter le parallélisme massif pour résoudre des problèmes complexes nécessitant une puissance de calcul importante.

Le traitement distribué présente de nombreux avantages : réduction du temps d'exécution, meilleure tolérance aux pannes, scalabilité et optimisation de l'utilisation des ressources. Dans le cadre de ce TP de Grilles Informatiques, nous implémentons un tri distribué de N éléments numériques à l'aide du simulateur GridSim.

L'objectif principal de ce travail est de concevoir, simuler et analyser une solution de tri parallèle basée sur une architecture maître-travailleurs, en respectant les exigences du sujet proposé par Mme S. SENHADJI.

2 Méthode de tri distribué

La méthode adoptée est un tri distribué de type *divide-and-conquer* suivi d'une fusion :

1. Le nœud maître génère une liste de N entiers aléatoires.
2. Cette liste est divisée en k sous-listes (où k est le nombre de travailleurs).
3. Chaque travailleur reçoit une sous-liste, la trie localement à l'aide de l'algorithme de tri de Java (`Collections.sort`).
4. Les sous-listes triées sont renvoyées au maître.
5. Le maître fusionne ces k listes triées en une seule liste finale triée en utilisant un algorithme de fusion *k-way* basé sur une file de priorité (`PriorityBlockingQueue`).

Cette approche combine la simplicité du tri local avec une fusion efficace en temps $O(N \log k)$.

3 Topologie de la grille

La topologie simulée est une **topologie en étoile**. Tous les nœuds travailleurs sont connectés directement au nœud maître via des liens réseau `SimpleLink` de GridSim.

Les paramètres réseau utilisés sont les suivants :

- Débit (*baud rate*) : 1 000 000 bits/seconde
- Délai de propagation : 10 ms
- MTU : 1500 octets

Cette topologie simplifie la communication tout en restant réaliste pour une petite grille de calcul.

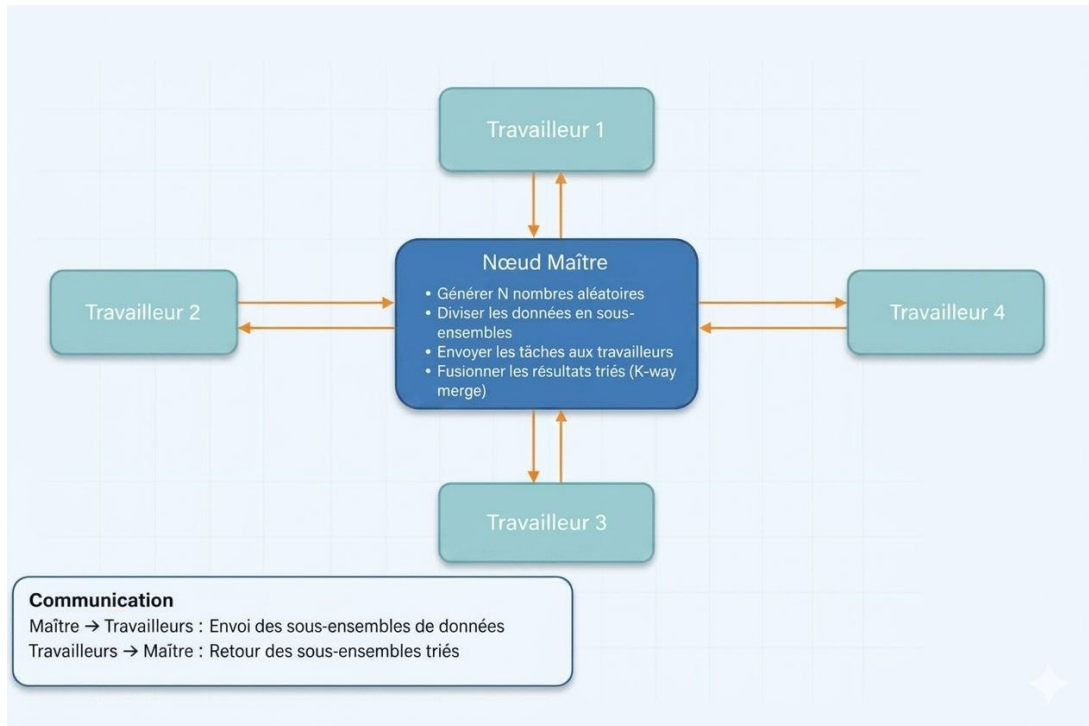


FIGURE 1 – Architecture de tri distribué basée sur le modèle Maître-Travailleurs simulée avec GridSim.

4 Rôle des entités

- **GridSim** : Simulateur de grille qui gère l'avancement du temps discret, les événements et la communication entre entités.
- **MasterNode** : Entité centrale responsable de la génération des données, de la répartition des sous-tâches, de la collecte des résultats partiels et de la fusion finale. Il coordonne l'ensemble du processus.
- **WorkerNode** : Nœuds de calcul qui reçoivent une sous-liste, effectuent le tri local et renvoient le résultat trié au maître. Chaque travailleur fonctionne de manière autonome et parallèle.

5 Stratégie de distribution et fusion

La distribution est réalisée de manière équitable : la liste originale de taille N est découpée en k sous-listes de taille approximativement N/k . Chaque sous-liste est encapsulée dans un objet `DataMessage` et envoyée au travailleur correspondant via l'événement `SCHEDULE_NOW`.

Après le tri local, chaque travailleur renvoie sa liste triée au maître. La fusion est effectuée à l'aide d'un algorithme *k-way merge* optimisé :

- Insertion des premiers éléments de chaque liste triée dans une file de priorité.
- Extraction répétée du plus petit élément et insertion du suivant de la même liste jusqu'à épuisement de toutes les listes.

Cette stratégie garantit une complexité temporelle optimale pour la fusion et une charge de travail équilibrée entre les travailleurs.

6 Résultats de simulation

Dans la simulation réalisée avec 4 travailleurs et $N = 40$ éléments, les données ont été correctement réparties. Chaque travailleur a reçu une sous-liste, l'a triée localement, puis renvoyée au maître.

Le maître a collecté les résultats partiels et effectué la fusion, produisant une liste finale entièrement triée et identique au résultat attendu d'un tri séquentiel.

L'observation des traces de simulation montre :

- ✓ Une parallélisation efficace du tri local.
- ✓ Une communication réseau simulée réaliste via les liens **SimpleLink**.
- ✓ Un temps de fusion négligeable grâce à l'algorithme basé sur une file de priorité.
- ✓ Une bonne scalabilité pour un petit nombre de nœuds.

Les sorties console colorées et les barres de progression confirment le bon déroulement séquentiel des étapes (initialisation, distribution, tri parallèle, fusion).

7 Discussion

L'approche maître-travailleurs présente plusieurs avantages : simplicité de mise en œuvre, bonne répartition de la charge de calcul et facilité de surveillance centrale. L'utilisation de GridSim permet de modéliser de manière réaliste les aspects réseau (délai, débit) sans nécessiter de matériel physique.

Cependant, certaines limitations existent :

- La topologie en étoile peut devenir un goulot d'étranglement pour un grand nombre de travailleurs.
- Le maître représente un point unique de défaillance.
- La simulation ne prend pas en compte les pannes de nœuds ni la tolérance aux fautes.

- Pour des très grands N , une approche hiérarchique ou peer-to-peer serait plus adaptée.

Des améliorations futures pourraient inclure l'ajout de mécanismes de tolérance aux fautes, une topologie plus complexe (arbre ou maillage) et l'intégration de données réelles de grande taille.

8 Conclusion

Ce TP a permis de mettre en œuvre avec succès un tri distribué dans un environnement de grille simulé avec GridSim. L'architecture maître-travailleurs en topologie étoile s'est révélée efficace pour répartir le calcul et fusionner les résultats.

Ce travail illustre concrètement les principes fondamentaux des grilles informatiques : distribution des tâches, parallélisme et coordination via un simulateur. Il ouvre la voie à des applications plus complexes telles que le tri de grands volumes de données ou d'autres algorithmes distribués (recherche, calcul scientifique, etc.).

L'utilisation de GridSim s'avère être un outil pédagogique puissant pour comprendre le fonctionnement des grilles informatiques sans infrastructure physique coûteuse.

Références

- [1] Buyya, R., Murshed, M. *GridSim : A Toolkit for the Modeling and Simulation of Distributed Resource Management and Scheduling for Grid Computing*. Concurrency and Computation : Practice and Experience, 2002.
- [2] Senhadji, S. *Support de cours - Grilles Informatiques*. Master 1 SID, Université des Sciences et de la Technologie d'Oran – Mohamed Boudiaf.